

RUES ANÁLISIS

Pablo Ibarlucea González y Oscar Javier Bachiller Sandoval

<https://github.com/pabloibargon/rues-analysis>

- **Integración:** Enriquecer los datos registrales con contexto macroeconómico (Banco Mundial).
- **Limpieza:** Estandarización de fechas y manejo de desbalanceo de clases.
- **Análisis No Supervisado:** Segmentación de empresas (Clustering).
- **Análisis Supervisado:** Predicción del tipo de sociedad (Clasificación).
- **Validación:** Contraste de hipótesis geográfico.

python

```
1 url = "https://api.worldbank.org/v2/en/country/COL?downloadformat=csv"
2 response = requests.get(url) # Descarga directa del ZIP
3
4 target_indicators = {
5     'NY.GDP.MKTP.KD.ZG': 'PIB_Crecimiento',
6     'FP.CPI.TOTL.ZG': 'Inflacion', ...
7 }
8
9 # Transformación: De formato 'Ancho' a 'Largo' y luego Pivot
10 # Objetivo: Tener [Año, PIB, Inflacion, ...]
11 wb_melted = wb_clean.melt(id_vars=['Indicador'], var_name='Year', ...)
12 wb_pivot = wb_melted.pivot(index='Year', columns='Indicador', values='Valor')
13
14 # Integración: Contexto de nacimiento (Año Matrícula)
15 df = df.merge(wb_pivot.add_suffix('_Matricula'), on='Year_Matricula', how='left')
```

Fuente Externa: API del Banco Mundial.

Variables Clave Agregadas:

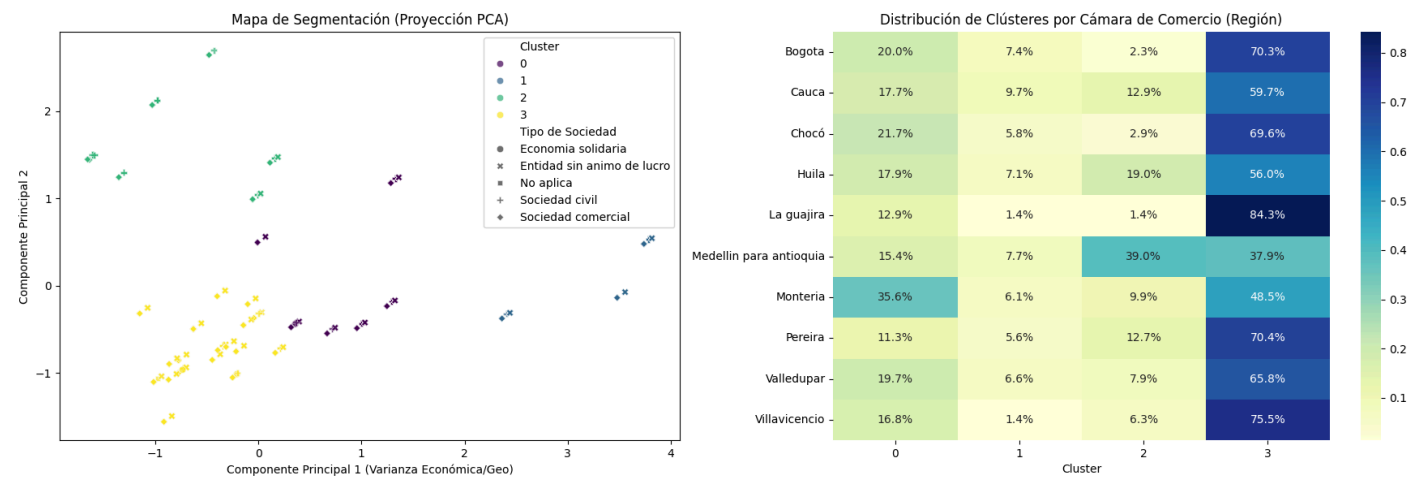
- Crecimiento del PIB anual.
- Inflación (IPC).
- Tasa de desempleo.

Estrategia: Left Join utilizando el **Año de Matrícula**.

python

```
1 # Preprocesamiento diferenciado numérico/categorico
2 preprocessor = ColumnTransformer(transformers=[
3     ('num', StandardScaler(), num_features),
4     ('cat', OneHotEncoder(), cat_features)
5 ])
6
7 # Pipeline: Limpieza -> PCA (Reducción) -> Modelo
8 pipeline = Pipeline([
9     ('preprocessor', preprocessor),
10    ('pca', PCA(n_components=2)),
11    ('kmeans', KMeans(n_clusters=4, random_state=44))
12 ])
13
14 pipeline.fit(df_model)
15
16 # Guardamos coordenadas PCA para visualización
17 df_model['Cluster'] = pipeline.named_steps['kmeans'].labels_
```

Modelo: K-Means (k=4) + PCA.

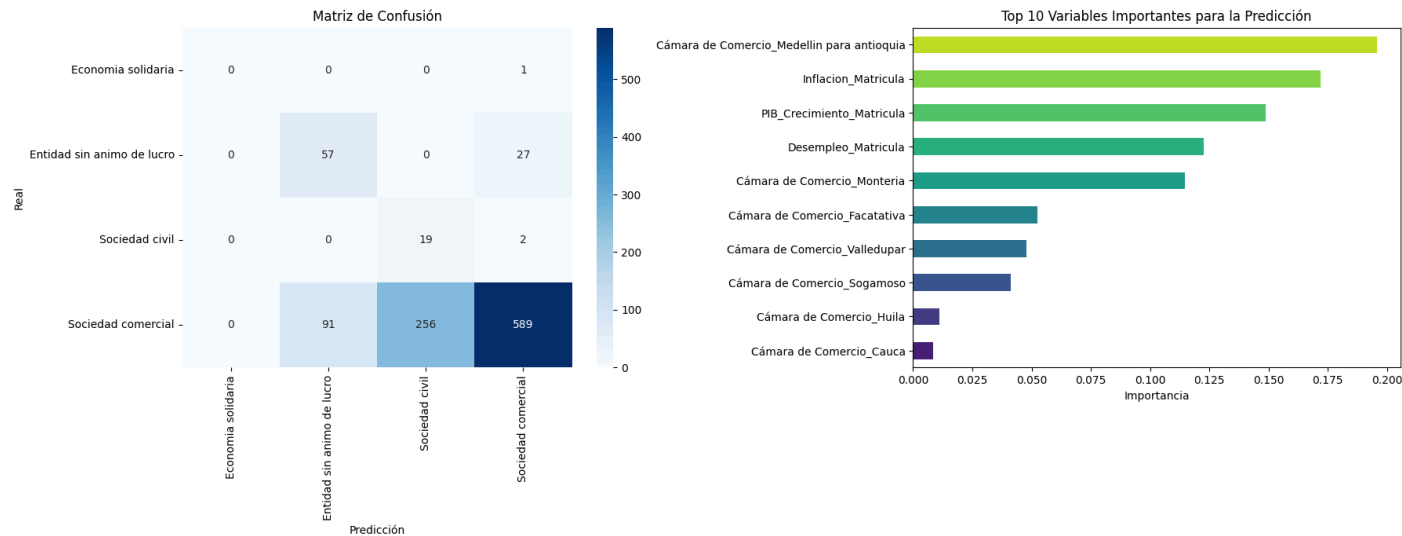


Conclusiones: Existencia de un “Cluster Base” (70% de datos) y una anomalía estructural en **Antioquia** (Cluster 2).

python

```
1 # 1. Filtro: Eliminar clases con < 2 registros (rompen el stratify)
2 v_counts = df_clf[target_col].value_counts()
3 df_clf = df_clf[df_clf[target_col].isin(v_counts[v_counts >= 2].index)]
4
5 # 2. Split estratificado (Mantiene proporción de clases)
6 X_train, X_test, y_train, y_test = train_test_split(..., stratify=y)
7
8 # 3. Random Forest con pesos balanceados
9 model_rf = RandomForestClassifier(
10     n_estimators=200,
11     class_weight="balanced", # CRÍTICO para datos desbalanceados
12     n_jobs=-1
13 )
14 model_rf.fit(X_train, y_train)
```

Modelo: Random Forest Classifier.

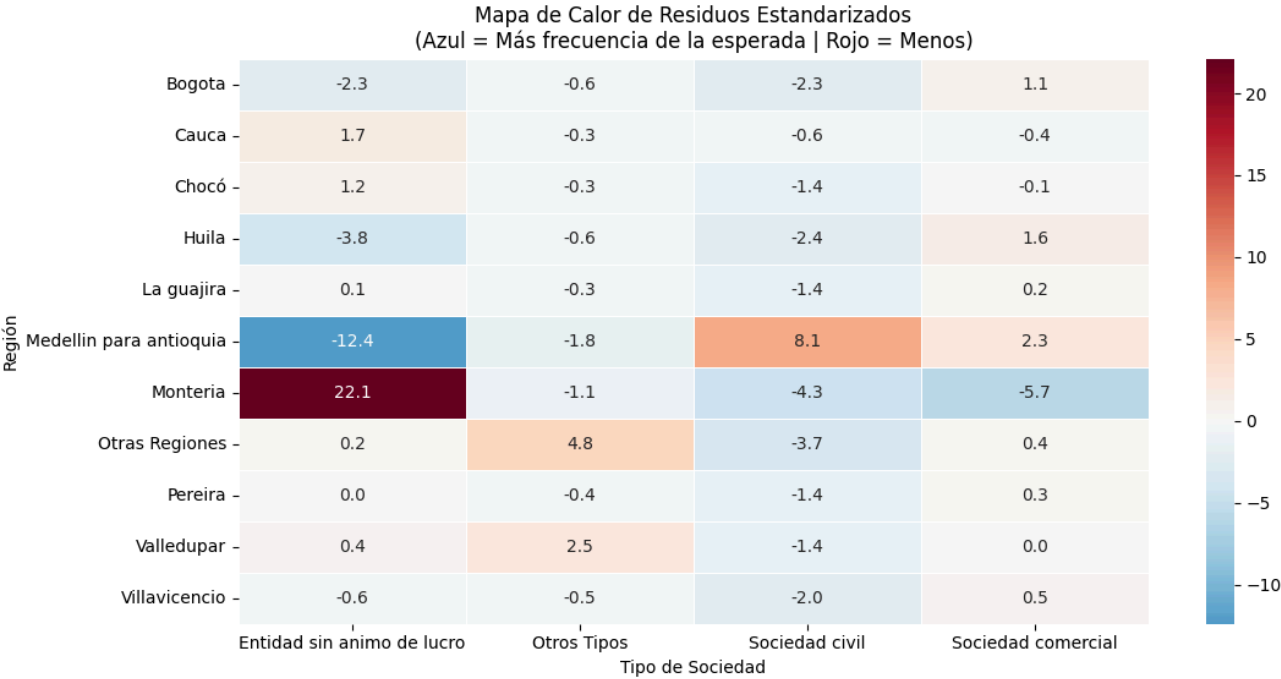


Conclusiones: La ubicación (Cámara de Comercio) y la **Inflación** al momento de registro son los predictores más potentes.

python

```
1 # Agrupación de colas largas para validez estadística
2 top_camaras = df['Cámara'].value_counts().nlargest(10).index
3 df['Region_Agrupada'] = df['Cámara'].apply(
4     lambda x: x if x in top_camaras else 'Otras Regiones'
5 )
6
7 # Tabla de contingencia: Región vs Tipo Sociedad
8 contingency_table = pd.crosstab(df['Region_Agrupada'], df['Sociedad_Agrupada'])
9
10 # Test estadístico
11 chi2, p_val, dof, expected = stats.chi2_contingency(contingency_table)
12
13 # Cálculo de Residuos Estandarizados (Para el Heatmap)
14 residuals = (contingency_table - expected) / np.sqrt(expected)
```

Prueba: Chi-Cuadrado de Pearson. **Hipótesis:** Dependencia entre Región y Tipo de Sociedad.



Resultado: $p < 0.05$. Se rechaza la independencia. Montería y Medellín muestran desviaciones significativas del comportamiento nacional.

- **Éxito en la Integración:** Las variables macroeconómicas demostraron ser señales predictivas reales, no ruido.
- **Singularidad Regional:** El tejido empresarial no es homogéneo; Antioquia y Montería requieren análisis diferenciados.
- **Desafío de Datos:** El desbalanceo masivo hacia “Sociedad Comercial” (90%) es el principal reto técnico.

Gracias